BitX UI/UX Design Specification

Executive Summary

The BitX platform provides a modern, responsive web interface built with React 19 and Tailwind CSS. The UI serves multiple user personas — from developers interacting with AI agents via chat, to administrators managing users and pipelines, to stakeholders monitoring system health and agent performance through a real-time dashboard.

The interface follows a **dark-mode-first design** with a sidebar navigation pattern, supporting 5 primary views: Chat (agent interaction), Local Agent Dashboard (AI engine monitoring), Pipelines (multi-agent workflows), Admin (user and agent management), and About (platform information).

---

1. Platform Overview

1.1 Application Suite

Application	Purpose	Technology
Agents Hub Client	Primary user interface (Port 3000)	React 19, Vite, Tailwind CSS
Agents Hub Server	API backend (Port 4001)	Fastify 5, TypeScript, SQLite
Product Hub UI	Legacy management interface (Port 4201)	Angular 17
Product Hub Backend	Legacy API (Port 8081)	Spring Boot 3.2.2

1.2 User Personas

Persona	Primary Views	Key Actions
Developer	Chat, Local Agents	Interact with agents, monitor runs, view tool outputs
Business Analyst	Chat, Pipelines	Submit analysis requests, review agent artifacts
QA Lead	Chat, Local Agents	Run test agents, review test results, approve changes
Admin	Admin, Local Agents	Manage users, agents, assignments; monitor system health
Stakeholder	Local Agents, Pipelines	Monitor agent activity, review dashboards

1.3 Technology Stack

Layer	Technology	Purpose
Framework	React 19	Component-based UI
State Management	Zustand	Lightweight, hook-based stores
Routing	React Router v6	Client-side navigation
Styling	Tailwind CSS	Utility-first CSS with dark mode
Build Tool	Vite	Fast dev server and bundler
HTTP Client	Axios	API communication

---

2. Information Architecture

2.1 Route Hierarchy

```
/login                          LoginPage (public)
/                               AppLayout (protected)
  ├── /                         ChatPage (default)
  ├── /pipelines                PipelineLayout
  ├── /admin                    AdminLayout (admin-only)
  ├── /local-agents             LocalAgentDashboard
  └── /about                    AboutPage
```

2.2 Component Hierarchy

```
BrowserRouter
  └── AuthInitializer
        ├── /login → LoginPage
        └── ProtectedRoute
              └── AppLayout
                    ├── Sidebar
                    ├── Header
                    └── <Outlet> (page content)
                          ├── ChatPage
                          │     ├── MessageBubble
                          │     ├── ChatInput
                          │     ├── SuggestionButtons
                          │     └── AgentProgressBar
                          ├── LocalAgentDashboard
                          ├── PipelineLayout
                          │     ├── PipelineList
                          │     ├── PipelineBuilder
                          │     └── PipelineRunner
                          └── AdminLayout
                                ├── AgentsPanel / AgentForm
                                ├── UsersPanel
                                ├── AssignmentsPanel
                                ├── UserJourneysPanel
                                └── AuditLog
```

2.3 Authentication Flow

```
App loads --> AuthInitializer --> getMe() API call
                                      |
                              +-----------+
                              |           |
                           Valid        Invalid
                              |           |
                        Set user      Clear auth
                        in store      redirect to /login
                              |
                        ProtectedRoute
                              |
                        Check isAuthenticated
                          /         \
                        Yes          No --> /login
                          |
                        AdminRoute (for /admin)
```

```
                          |
                        Check role === 'admin'
                          /         \
                        Yes          No --> /
```

---

3. Layout System

3.1 AppLayout Structure

```
+-----------------------------------------------+
|                  Header                       |
+----------+-----------------------------------+
|          |                                   |
| Sidebar  |           Main Content            |
|          |           (<Outlet>)              |
| - Nav    |                                   |
| - Agents |                                   |
| - WorkDir|                                   |
|          |                                   |
+----------+-----------------------------------+
```

3.2 Sidebar Navigation

The sidebar provides:

- **Navigation links**: Chat, Pipelines, Local Agents, Admin, About
- **Agent list**: Scrollable list of available agents for the Chat view
- **Working directory selector**: Choose the filesystem scope for agent operations

3.3 Header

The header displays:

- **User information**: Username and role badge
- **Theme toggle**: Light/dark mode switch
- **System status**: Quick health indicator

3.4 Responsive Design

Breakpoint	Layout
Mobile (< 640px)	Sidebar collapsed, single-column content
Tablet (640-1024px)	Sidebar visible, 2-column grids
Desktop (> 1024px)	Full sidebar, 3-column grids

3.5 Dark Mode

All components support dark mode via Tailwind's `dark:` prefix. The design defaults to a **slate-based dark palette**:

- Background: `bg-white` / `dark:bg-slate-800`
- Borders: `border-slate-200` / `dark:border-slate-700`
- Text: `text-slate-900` / `dark:text-slate-100`

---

4. Chat Interface

4.1 ChatPage Architecture

The Chat view is the primary interaction surface for working with AI agents.

```
+-----------------------------------------+
|  Agent Selector / Context Bar            |
+-----------------------------------------+
|                                          |
|  +-------------------------------------+ |
|  |        Message Bubbles               | |
|  |  [User bubble - right aligned]       | |
|  |  [Agent bubble - left aligned]       | |
|  |  [Agent bubble with code block]      | |
|  |                                      | |
|  +-------------------------------------+ |
|                                          |
|  +-------------------------------------+ |
|  |  AgentProgressBar (during runs)      | |
|  +-------------------------------------+ |
|                                          |
|  +-------------------------------------+ |
|  |  SuggestionButtons                   | |
|  +-------------------------------------+ |
|                                          |
|  +-------------------------------------+ |
|  |  ChatInput                           | |
|  +-------------------------------------+ |
+-----------------------------------------+
```

4.2 Key Components

Component	Purpose
MessageBubble	Renders user (right) and assistant (left) messages with markdown and code highlighting
ChatInput	Text input with submit button, keyboard shortcuts
SuggestionButtons	Pre-built prompt suggestions contextual to the selected agent
AgentProgressBar	Real-time progress indicator showing current orchestrator stage
WorkingDirectorySelector	Scopes agent file operations to a specific directory

---

5. Local Agent Dashboard

The dashboard is the monitoring and management center for the AI engine.

5.1 Dashboard Layout

```
+-----------------------------------------------------------------+
|  Local Agent Dashboard                                           |
|  LM Studio runtime · {N} profiles loaded    [Discover] [Refresh]  |
+-----------------------------------------------------------------+
|                                                                   |
|  +------------------+  +------------------+  +------------------+  |
|  | System Health    |  | Queue Status     |  | Run Statistics   |  |
```

```
|  |                  |  |                  |  |                  |  |
|  | Status: healthy  |  | Queued: 0        |  | Total: 47        |  |
|  | Router: avail    |  | Active: 1/2      |  | Completed: 42    |  |
|  | Worker: avail    |  | [profile_id]     |  | Failed: 3        |  |
|  |                  |  |                  |  | In Progress: 2   |  |
|  +------------------+  +------------------+  +------------------+  |
|                                                                      |
|  +-------------------------------------------------------------+ |
|  | Queued Runs (if any)                                         | |
|  | #1 | process_analyst | cuid... | [Cancel]                   | |
|  +-------------------------------------------------------------+ |
|                                                                      |
|  +-------------------------------------------------------------+ |
|  | Recent Runs                                                   | |
|  | Profile     | Status    | Prompt         | Tokens  | Time     | |
|  |-------------|-----------|----------------|---------|----------| |
|  | backend_dev | completed | Add health...  | 12,345  | 2 min ago| |
|  | qa_lead     | failed    | Create test... | 8,901   | 5 min ago| |
|  +-------------------------------------------------------------+ |
|                                                                      |
|  +-------------------------------------------------------------+ |
|  | Agent Profiles                                                | |
|  | +--------+ +--------+ +--------+                            | |
|  | |PA-     | |BA-     | |ARCH-   |                            | |
|  | |Process | |Business| |Solution|                            | |
|  | |Analyst | |Analyst | |Archit. |                            | |
|  | |discover| |discover| |design  |                            | |
|  | |router  | |router  | |router  |                            | |
|  | +--------+ +--------+ +--------+                            | |
|  +-------------------------------------------------------------+ |
+-----------------------------------------------------------------+
```

5.2 Status Badge System

Status	Color	Use
`healthy` / `completed`	Green (`bg-green-100 text-green-800`)	Success states
`failed` / `error`	Red (`bg-red-100 text-red-800`)	Failure states
`running` / `in_progress`	Purple (`bg-purple-100 text-purple-800`)	Active states
`queued`	Blue (`bg-blue-100 text-blue-800`)	Waiting states
`degraded`	Orange (`bg-orange-100 text-orange-800`)	Warning states
`cancelled`	Slate (`bg-slate-100 text-slate-600`)	Neutral states

All badges include dark mode variants using `dark:bg-{color}-900/30 dark:text-{color}-400`.

5.3 Run Detail Modal

Clicking a run in the Recent Runs table opens a modal overlay:

```
+---------------------------------------------------+
| [X]                                               |
| profile_id                    Status Badge        |
| Run ID: cuid...                                   |
```

```
+-----------------------------------------------+
| Prompt:                                       |
| "Create a test plan for authentication..."    |
+-----------------------------------------------+
| Tokens: 12,345 | Duration: 45.2s | Created: ...  |
+-----------------------------------------------+
| Output:                                       |
| [scrollable pre-formatted text area]          |
+-----------------------------------------------+
| Steps:                                        |
| 1. intake | router | 2.3s                     |
| 2. context_gathering | router | 5.1s          |
| 3. planning | router | 3.8s                  |
| 4. tool_call: search_files | 0.5s             |
| 5. tool_call: read_file | 0.3s                |
| 6. finalize | router | 1.2s                  |
+-----------------------------------------------+
| Artifacts:                                    |
| [plan] execution-plan                         |
| [summary] run-summary                         |
+-----------------------------------------------+
```

5.4 Data Refresh Strategy

Data	Refresh Interval	Method
Health, Queue, Runs, Profiles	10 seconds	`setInterval` + `Promise.all`
Run Detail	On-demand	Click to load
Model Discovery	Manual button	User-triggered

---

6. Pipeline Builder

6.1 Pipeline Views

View	Component	Purpose
Pipeline List	`PipelineList.tsx`	Browse and select saved pipelines
Pipeline Builder	`PipelineBuilder.tsx`	Design multi-agent workflow sequences
Pipeline Runner	`PipelineRunner.tsx`	Execute and monitor pipeline runs

6.2 Multi-Agent Workflow Design

The Pipeline Builder allows users to chain agents in sequence, defining:

- **Agent sequence**: Which agents run in what order
- **Handoff data**: What outputs from one agent feed into the next
- **Gate conditions**: When to proceed to the next agent

---

7. Admin Panel

7.1 Admin Sections

Section	Component	Capabilities
Agents	`AgentsPanel.tsx` + `AgentForm.tsx`	CRUD operations on agent definitions
Users	`UsersPanel.tsx`	User management (create, edit, delete, role assignment)
Assignments	`AssignmentsPanel.tsx`	Assign agents to users or teams
User Journeys	`UserJourneysPanel.tsx`	Track user journey definitions
Audit Log	`AuditLog.tsx`	System activity and change history

7.2 Access Control

- Admin panel is wrapped in `AdminRoute` which checks `user.role === 'admin'`
- Non-admin users are redirected to the home page
- All admin actions go through authenticated API endpoints

---

8. State Management

8.1 Zustand Auth Store

```
authStore {
  isAuthenticated: boolean
  user: { userId, username, role }
  setUser(user): void
  logout(): void
}
```

8.2 Local Agent API Client

The API client provides 15 typed functions:

Category	Functions
Health	`getLocalAgentHealth()`, `discoverModels()`
Profiles	`getProfiles()`, `getProfile(id)`
Runs	`getRuns(limit)`, `getRunDetail(runId)`, `createRun(params)`
Run Details	`getRunSteps(runId)`, `getRunArtifacts(runId)`, `getRunApprovals(runId)`
Queue	`getQueueStatus()`, `cancelQueuedRun(runId)`
Approvals	`resolveApproval(id, decision)`
Routing	`routePrompt(prompt)`
Tools	`getToolDefinitions()`

8.3 Data Fetching Pattern

```
Component mounts --> loadData()
                        |
                  Promise.all([
                    getLocalAgentHealth(),
                    getRuns(),
```

```
                  getQueueStatus(),
                  getProfiles()
                ])
                      |
                setState(results)
                      |
                setInterval(loadData, 10_000)
```

---

9. Real-Time Updates

9.1 Polling Strategy

The dashboard uses **10-second polling** for data freshness:

- Health status refreshes to detect model changes
- Queue status refreshes to show new/completed runs
- Run list refreshes to display status transitions
- Profile count refreshes to detect hot-reloaded profiles

9.2 Progress Callbacks

The Orchestrator emits real-time progress events through the backend:

```
{
  "runId": "abc123",
  "stage": "tool_loop",
  "message": "Executing tool: read_file",
  "data": { "tool": "read_file" }
}
```

These propagate to the `AgentProgressBar` component in the Chat view.

9.3 Optimistic UI Patterns

Action	Optimistic Behavior
Cancel run	Button disabled immediately, confirmation after API response
Discover models	Button shows "Discovering..." state during API call
Refresh	"Refreshing..." overlay prevents double-clicks

---

10. Design System

10.1 Color Palette

Role	Light Mode	Dark Mode
Background	`white`	`slate-800` / `slate-900`
Surface	`slate-50`	`slate-800`
Border	`slate-200`	`slate-700`
Primary text	`slate-900`	`slate-100`
Secondary text	`slate-500`	`slate-400`

Primary accent	`blue-600`	`blue-400`
Success	`green-600`	`green-400`
Error	`red-600`	`red-400`
Warning	`orange-600`	`orange-400`
Active	`purple-600`	`purple-400`

10.2 Typography

Use	Class	Example
Page title	`text-2xl font-bold`	"Local Agent Dashboard"
Section heading	`text-lg font-semibold`	"Recent Runs"
Body text	`text-sm`	Profile descriptions
Metadata	`text-xs text-slate-500`	Timestamps, IDs
Monospace	`font-mono text-xs`	Run IDs, file paths

10.3 Card Components

```
Border: border border-slate-200 dark:border-slate-700
Background: bg-white dark:bg-slate-800
Radius: rounded-lg or rounded-xl
Padding: p-4 or p-6
Shadow: (minimal, relies on borders)
```

10.4 Interactive States

State	Style
Hover	`hover:bg-slate-50 dark:hover:bg-slate-700/50`
Active	`bg-blue-50 dark:bg-blue-900/20`
Disabled	`opacity-50 cursor-not-allowed`
Focus	Standard browser focus ring

10.5 Spacing System

Use	Values
Component gap	`gap-4`, `gap-6`
Internal padding	`p-3`, `p-4`, `p-6`
Section spacing	`space-y-4`, `space-y-6`
Badge padding	`px-2 py-0.5`

---

11. Component Inventory

Directory	Components	Purpose
`components/chat/`	ChatPage, MessageBubble, ChatInput, SuggestionButtons, AgentProgressBar	Agent interaction
`components/local-agents/`	LocalAgentDashboard	AI engine monitoring
`components/layout/`	AppLayout, Header, Sidebar, AgentProcessPanel, WorkingDirectorySelector	Application shell
`components/pipelines/`	PipelineLayout, PipelineList, PipelineBuilder, PipelineRunner	Multi-agent workflows
`components/admin/`	AdminLayout, AgentsPanel, AgentForm, UsersPanel, AssignmentsPanel, UserJourneysPanel, AuditLog	Administration
`components/auth/`	LoginPage	Authentication
`components/about/`	AboutPage	Platform information

Total: 24 React components across 7 feature areas.

Cross-references: AI Engine Architecture (Section 10: API Surface) | Dashboards (Section 8: Implementation Specs)